Университет ИТМО

Отчёт по домашнему заданию 4

Обеспечение отказоустойчивости средствами Apache ZooKeeper

Студент: Смольникова Юлия Юрьевна

Группа: J3311

2026 г.

Содержание

1 Цель и окружение **2**

2 Проверка ансамбля ZooKeeper **2**

2.1 Запуск и роли серверов 2

2.2 Остановка одного сервера 3

2.3 Остановка двух серверов и восстановление 4

2.4 Эфемерный узел 5

3 Приостановка лидера без fencing **7**

4 Приостановка лидера с fencing по эпохе **7**

5 Сравнение с ДЗ 3 **9**

6 Ответы на вопросы **9**

6.1 Почему три сервера, а не два или четыре? 9

6.2 Почему защиту по эпохе проверяет получатель записи? 9

7 Итог **9**

1 Цель и окружение

Цель работы — развернуть ансамбль Apache ZooKeeper, проверить работу кворума и эфемерных узлов, а затем обеспечить восстановление координатора вычисления из ДЗ 3 после потери лидера. Отдельно рассматриваются публикация без защиты и публикация с защитой по эпохе.

Работа выполнялась в Windows с использованием PowerShell и Docker Desktop. Ансамбль состоит из трёх контейнеров образа `zookeeper:3.9.6`. На хосте их клиентские порты — `2181`, `2182` и `2183`. Для каждого сервера заданы собственный `ZOO_MY_ID` и отдельные тома данных и журнала; список участников `ZOO_SERVERS` одинаков. В качестве S3-совместимого хранилища использовался RustFS.

Входной объект — `raw/vacancies.csv`, его размер составил 847 619 384 байта. Координатор выполняет вычисление из ДЗ 3: отбирает вакансии по правилам DevOps, нормализует навыки и подсчитывает частоты сочетаний из одного, двух и трёх навыков. Данные читаются блоками, а промежуточные результаты каждого блока сохраняются в S3 как чекпоинты. В обоих экспериментах с координаторами тайм-аут сессии ZooKeeper составлял 10 секунд.

Все времена в таблицах экспериментов ниже приведены в UTC. В листингах показаны существенные строки фактического вывода; полные журналы находятся в каталоге `logs/` репозитория. На рисунках представлены скриншоты проверок.

2 Проверка ансамбля ZooKeeper

2.1 Запуск и роли серверов

Состояние контейнеров и роли проверялись командами:

```
docker compose up -d
docker compose ps
docker compose exec zk1 zkServer.sh status
docker compose exec zk2 zkServer.sh status
docker compose exec zk3 zkServer.sh status
```

В одной из проверок все три контейнера имели статус `Up`; роли были следующими:

Сервер	Порт хоста	Роль
zk1	2181	follower
zk2	2182	follower
zk3	2183	leader

Роль конкретного сервера после переизбрания может измениться. Конфигурация ансамбля сохранена в `compose.yaml`.

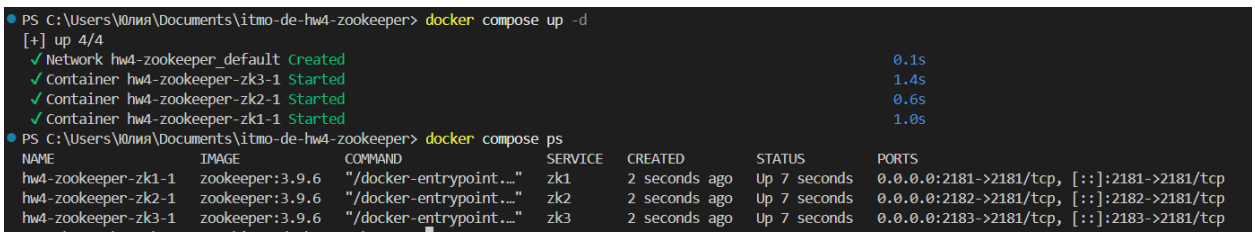


Рис. 1: Запуск трёх контейнеров ZooKeeper и их состояние

На рисунках ниже показана предварительная проверка ролей в отдельном запуске ансамбля: первые два сервера были последователями, третий — лидером.

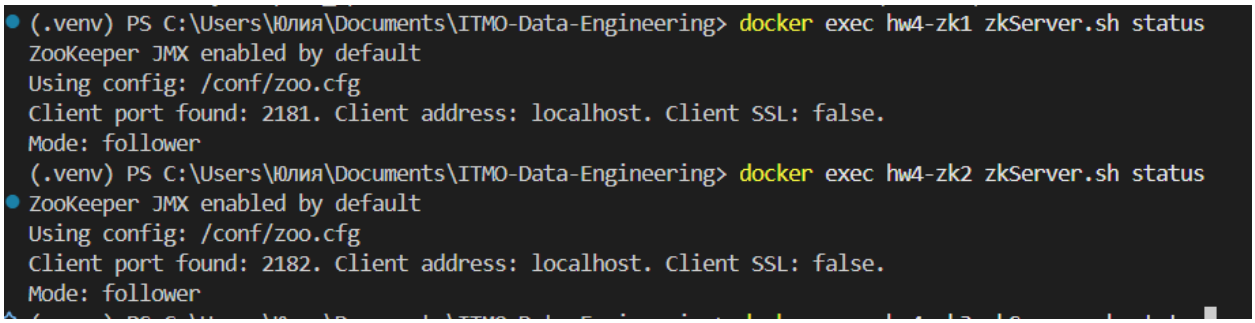


Рис. 2: Предварительная проверка: роли `zk1` и `zk2`

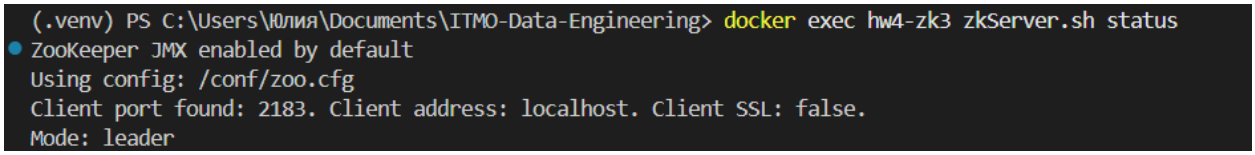


Рис. 3: Предварительная проверка: `zk3` является лидером

2.2 Остановка одного сервера

После остановки `zk2` в работе остались `zk1` и `zk3`; они сохранили кворум. Проверены роли и операция записи через `zk1`:

```
docker compose stop zk2
docker compose ps
docker compose exec zk1 zkServer.sh status
docker compose exec zk3 zkServer.sh status
$commands = @(
  'create /one-node-check-20261109 one-server-down-ok',
  'get /one-node-check-20261109',
  'delete /one-node-check-20261109',
  'quit'
)
$commands | docker compose exec -T zk1 timeout 20s zkCli.sh -server localhost:2181
```

На момент проверки `zk1` был последователем, `zk3` — лидером. В выводе клиента присутствовали строки:

```
Created /one-node-check-20261109
one-server-down-ok
```

Тестовый узел затем удалён командой `delete`. Журнал: `logs/zk/08_one_node_down_ensemble_works.txt`.

На рисунке показана другая проверка одного отказа: был остановлен прежний лидер `zk3`, после чего лидером стал `zk2`.

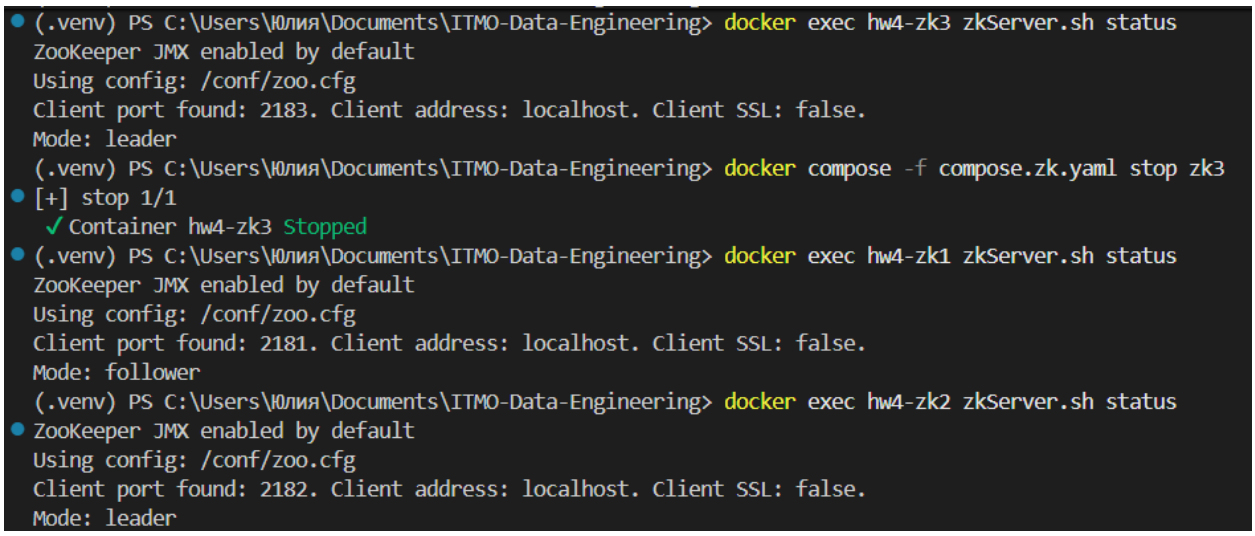


Рис. 4: Выбор нового лидера после остановки `zk3`

2.3 Остановка двух серверов и восстановление

Во второй проверке остановлены `zk2` и `zk3`; `docker compose ps` показывал работающим только `zk1`. Попытка выполнить запись через оставшийся узел завершилась ошибкой соединения:

```
docker compose stop zk2 zk3
docker compose ps
$commands = @('create /quorum-check-20261109 test', 'quit')
$commands | docker compose exec -T zk1 timeout 35s zkCli.sh -server localhost:2181

KeeperErrorCode = ConnectionLoss for /quorum-check-20261109
```

При одном работающем сервере большинство из трёх участников недоступно, поэтому запись не была подтверждена. После проверки серверы возвращены в работу:

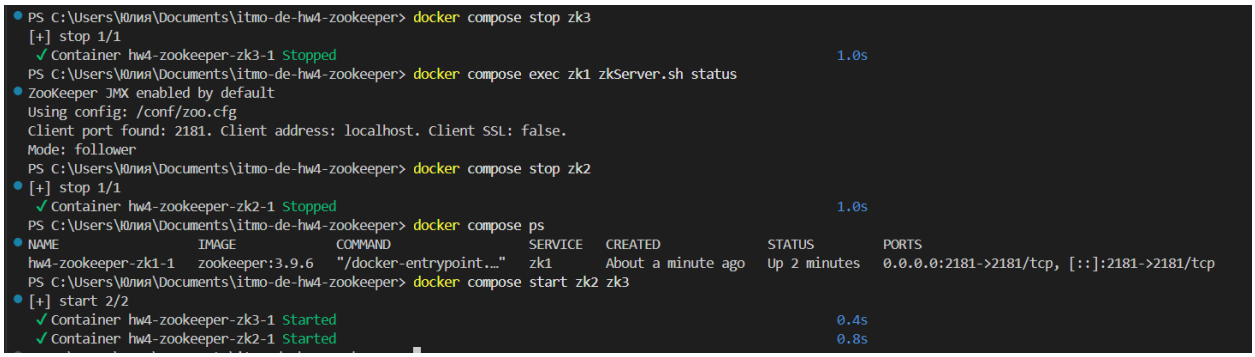


Рис. 5: Остановка двух серверов: в списке работающих контейнеров остался `zk1`

```
docker compose start zk2 zk3
docker compose ps
docker compose exec zk1 zkServer.sh status
docker compose exec zk2 zkServer.sh status
docker compose exec zk3 zkServer.sh status
```

Все три контейнера снова имели статус `Up`; наблюдались один лидер и два последователя. Выводы команд сохранены в `logs/zk/08_two_nodes_no_quorum.txt` и `logs/zk/09_two_nodes_restored.txt`.

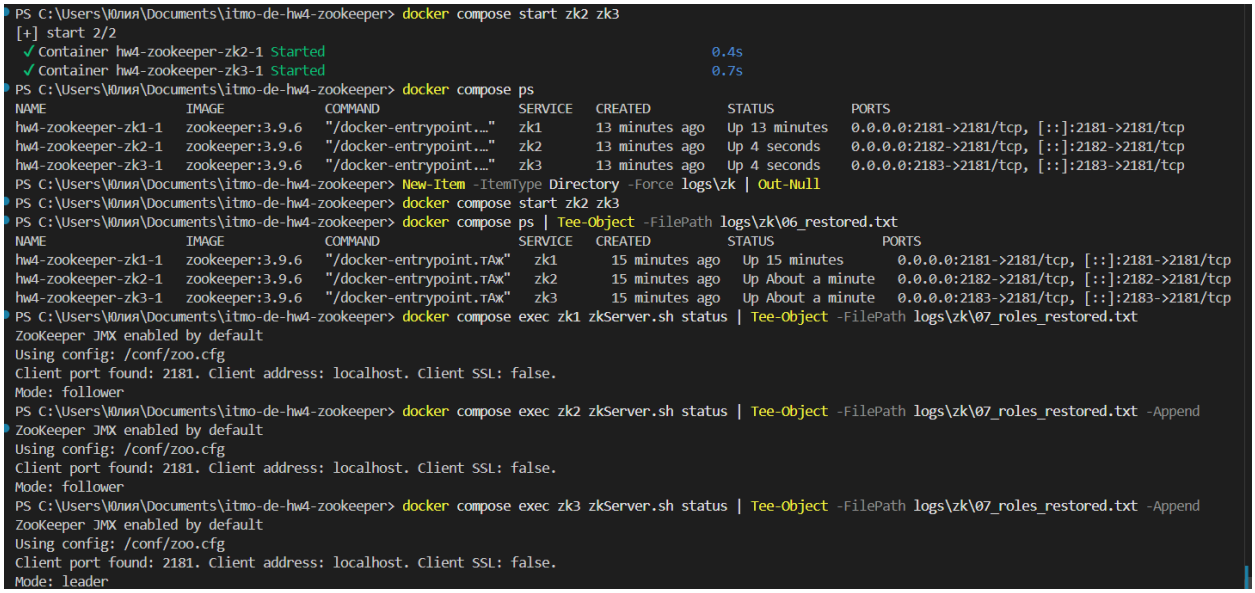


Рис. 6: Восстановление ансамбля и проверка ролей серверов

2.4 Эфемерный узел

На рисунках приведена предварительная проверка узла `/hw4-session`: его создание и чтение в первой сессии, чтение из второй сессии, закрытие первой сессии и проверка отсутствия узла.

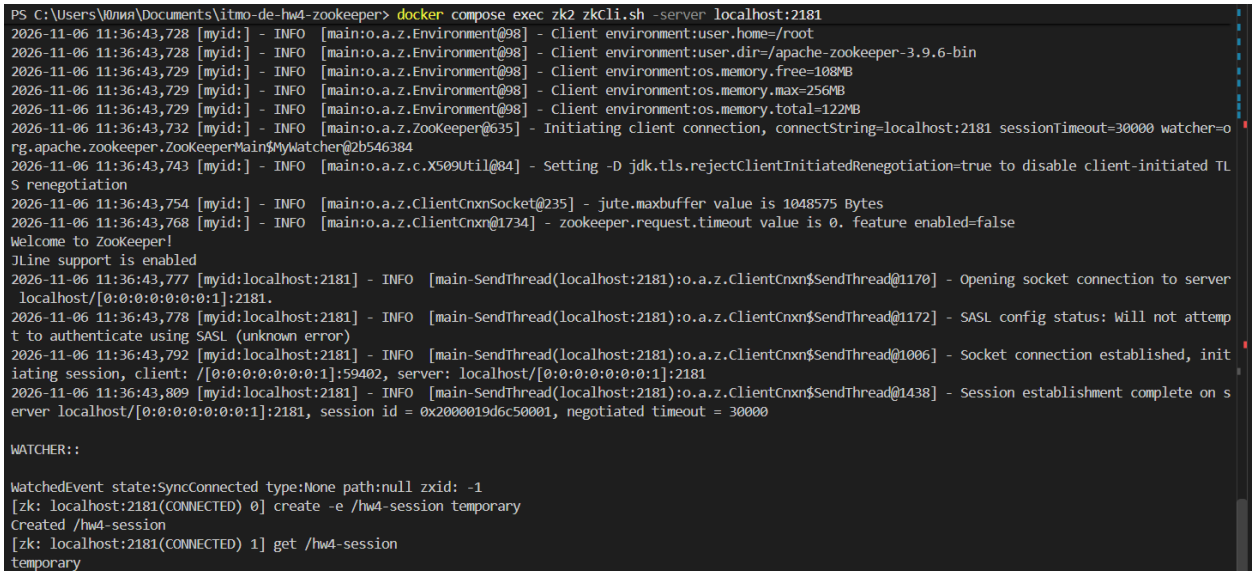


Рис. 7: Создание и чтение эфемерного узла в первой сессии

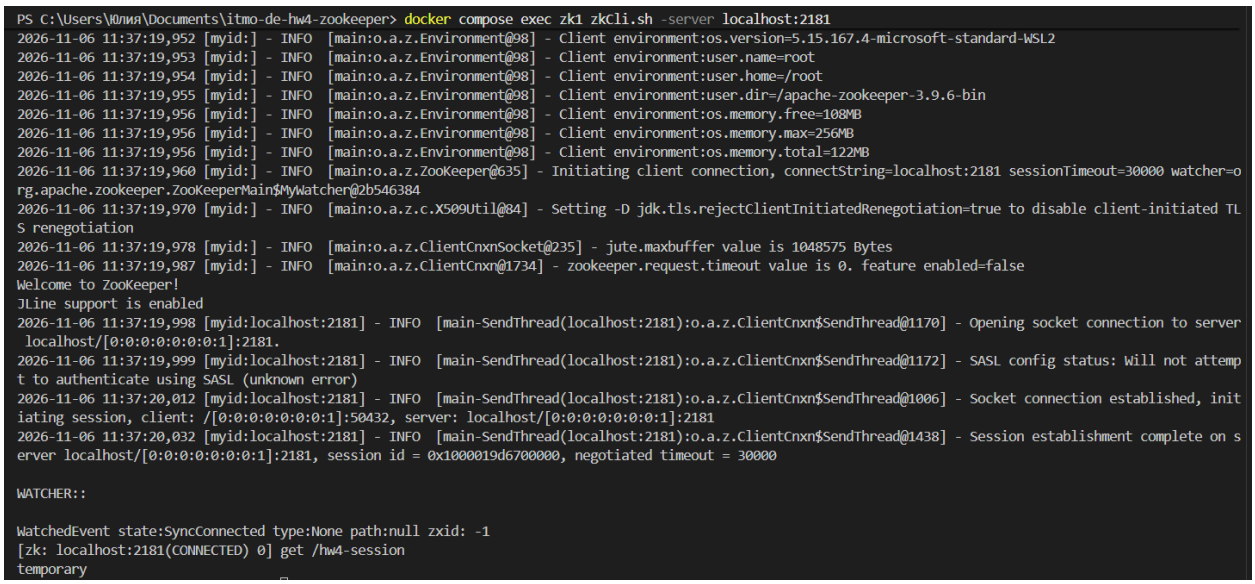


Рис. 8: Чтение эфемерного узла из второй клиентской сессии

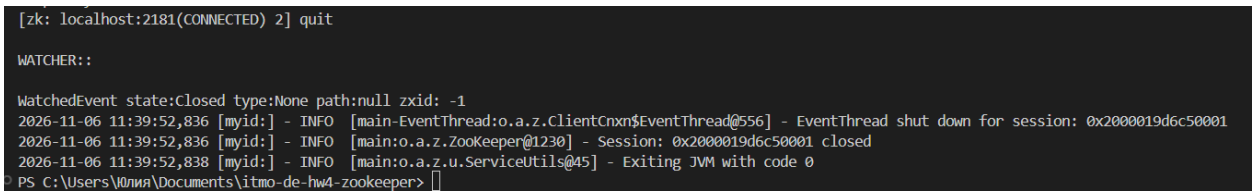


Рис. 9: Закрытие сессии, в которой был создан эфемерный узел

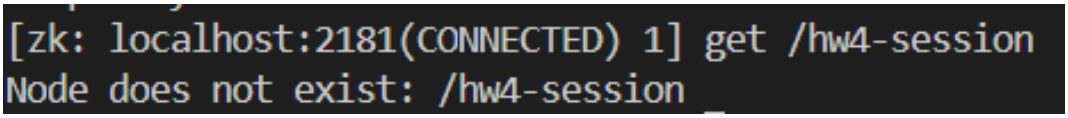


Рис. 10: Проверка отсутствия узла после закрытия сессии

Ниже приведён вывод повторной проверки с узлом `/hw4-ephemeral-20261109`. В первой сессии клиента он был создан и прочитан, после чего сессия закрыта командой `quit`:

```
$commands = @(
  'create -e /hw4-ephemeral-20261109 ephemeral-value',
  'get /hw4-ephemeral-20261109',
  'quit'
)
$commands | docker compose exec -T zk1 timeout 20s zkCli.sh -server localhost:2181

Created /hw4-ephemeral-20261109
ephemeral-value
WatchedEvent state:Closed
```

Во второй сессии тот же путь уже отсутствовал:

```
$commands = @('get /hw4-ephemeral-20261109', 'quit')
$commands | docker compose exec -T zk1 timeout 20s zkCli.sh -server localhost:2181
```

```
Node does not exist: /hw4-ephemeral-20261109
```

Исчезновение узла связано с закрытием сессии, которой он принадлежал. Журналы: `logs/zk/11_ephemeral_node_session.txt` и `logs/zk/12_ephemeral_node_absent.txt`.

3 Приостановка лидера без fencing

Для запуска `unfenced-20261106-09` использовались два координатора. Первый стал лидером с эпохой 1, второй ожидал его в очереди выборов. Лидер был приостановлен после сохранения чекпоинтов и возобновлён после окончания расчёта запасным координатором.

```
.\scripts\run_unfenced_demo.ps1 -RunId unfenced-20261106-09 -PauseSeconds 45
```

Событие	Время, UTC
Приостановка первого лидера	13:25:39.340
Запасной стал лидером	13:25:49.255
Запасной завершил расчёт	13:26:14.876
Первый лидер возобновлён	13:26:24.341

Время от приостановки до избрания нового лидера составило $49{,}255 - 39{,}340 = 9{,}915$ секунды. Запасной координатор обнаружил два сохранённых чекпоинта и пропустил повторный map для блоков 0 и 1.

```
became leader epoch=2
restored_checkpoints=2
checkpoint restored chunk=0; skipping map
checkpoint restored chunk=1; skipping map
wrote result coordinator=coordinator-2 epoch=2 pointer_version=1
wrote result coordinator=coordinator-1 epoch=1 pointer_version=2
```

После возобновления старый лидер продолжил вычисление и повторно обновил указатель результата. Его версия стала равна 2: без fencing получатель указателя принял обе записи. Журналы находятся в `logs/hw4/unfenced-20261106-09/`.

4 Приостановка лидера с fencing по эпохе

Для запуска `fenced-20261109-02` использовался тот же сценарий отказа с включённым `FENCING=1`. Сценарий ожидал завершения запасного координатора перед возобновлением прежнего лидера:

```
.\scripts\run_fenced_demo.ps1 -RunId fenced-20261109-02 -PauseSeconds 45
```

Событие	Время, UTC
Приостановка первого лидера	14:13:02.663
Запасной стал лидером	14:13:12.926
Запасной завершил расчёт	14:14:13.147
Первый лидер возобновлён	14:14:13.191

Время смены лидера составило 12,926 − 2,663 = 10,263 секунды, длительность паузы — 70,528 секунды. Запасной координатор восстановил два чекпоинта и завершил расчёт до возобновления старого лидера.

```
captured fencing token epoch=1 fence_version=1
became leader epoch=2
captured fencing token epoch=2 fence_version=2
restored_checkpoints=2
checkpoint restored chunk=0; skipping map
checkpoint restored chunk=1; skipping map
wrote result coordinator=coordinator-2 epoch=2 pointer_version=1
result write rejected coordinator=coordinator-1 epoch=1 error=BadVersionError
    expected_fence_version=1
pointer_version=1 coordinator_id=coordinator-2 epoch=2
registered_fence_epoch=2 fence_version=2
```

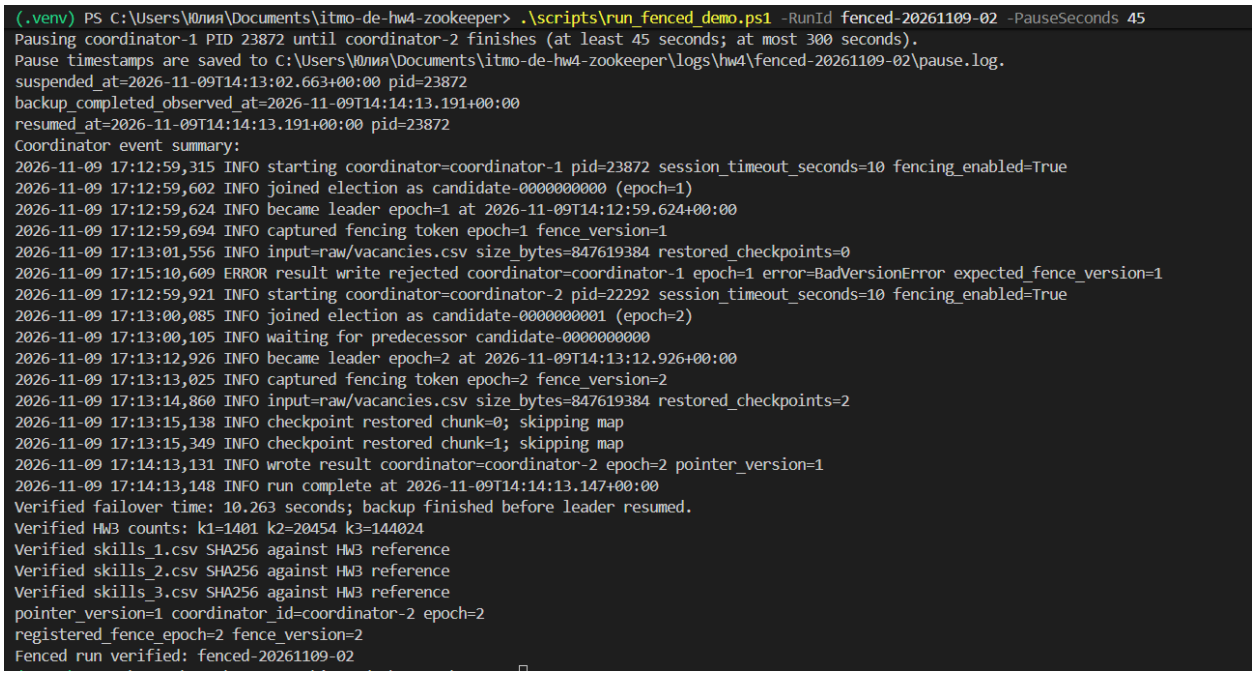


Рис. 11: Результат запуска с fencing: восстановление с чекпоинта и отклонение устаревшей записи

При избрании нового лидера его эпоха регистрируется в узле `fence`. При публикации проверка версии этого узла и обновление указателя результата выполняются одной транзакцией ZooKeeper. Старая версия токена больше не совпала с зарегистрированной, поэтому запись была отклонена с `BadVersionError`; указатель остался у запасного координатора.

Файлы расчёта сохраняются в отдельных S3-префиксах координаторов до публикации указателя. Поэтому утверждение об одном итоговом результате относится к *единственному действующему указателю*: защита указателя сама по себе не удаляет отдельные S3-объекты старого координатора. Журналы запуска сохранены в `logs/hw4/fenced-20261109-02/`.

5 Сравнение с ДЗ 3

В обоих запусках запасной координатор обработал 102 блока и получил одинаковые контрольные показатели: 229 389 прочитанных строк, 5 158 строк после отбора и 3 534 строки с навыками. Для каждого k приведены количество уникальных сочетаний и сумма их частот:

Размер сочетания k	Уникальные сочетания	Сумма частот
1	1 401	28 336
2	20 454	134 651
3	144 024	506 549

Для каждого запуска файлы `skills_1.csv`, `skills_2.csv` и `skills_3.csv` запасного координатора сравнены по SHA256 с эталонными файлами ДЗ 3 из `data/reference/results_v2/`. Все шесть сравнений совпали.

6 Ответы на вопросы

6.1 Почему три сервера, а не два или четыре?

Для принятия решений ZooKeeper требуется большинство голосующих серверов. В ансамбле из трёх большинство составляет два: один отказ можно пережить, сохранив два работающих узла. Из двух серверов для большинства также нужны два, поэтому отказ любого из них лишает ансамбль кворума. В ансамбле из четырёх большинство составляет три: он тоже выдерживает только один отказ, но требует дополнительный сервер. Для отказоустойчивости к двум отказам потребуется большинство из пяти серверов.

6.2 Почему защиту по эпохе проверяет получатель записи?

При сетевом сбое или длительной паузе старый лидер может не сразу узнать, что его сессия завершилась и выбран другой лидер. Поэтому одной проверки со стороны отправителя перед началом работы недостаточно: к моменту записи его право могло устареть. Получатель защищённого ресурса должен атомарно сопоставить токен записи с актуальным и отклонить старый токен.

В этой работе защищаемый ресурс — указатель результата в ZooKeeper. Логика публикации регистрирует эпоху, а ZooKeeper выполняет атомарную проверку версии узла `fence` и обновление указателя в одной транзакции. Сам по себе ZooKeeper не запрещает произвольную запись в независимое S3-хранилище: если требуется защищать также запись непосредственно в S3 или другую внешнюю систему, проверку токена должен обеспечивать получатель соответствующей записи.

7 Итог

Проверены работа ансамбля при потере одного сервера, потеря кворума при остановке двух серверов и удаление эфемерного узла после закрытия сессии. В обоих экспериментах запасной координатор продолжил вычисление с двух чекпоинтов, а результаты совпали

с ДЗ 3. Без fencing старый лидер повторно обновил указатель; с fencing его запись была отклонена, и указатель остался у координатора новой эпохи.